

PhishVN Edge: Software for benchmarking and deploying Vietnamese phishing detectors on low-cost edge devices

Thai Nguyen Vu^a

^a*Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam*

Abstract

PhishVN Edge provides scripts for exporting and benchmarking phishing classifiers on Linux edge devices. The standalone release combines scikit-learn-to-ONNX export, ONNX Runtime CPU benchmarking, a synthetic demonstration model, and article-asset generation from aggregate results. A historical NVIDIA Jetson Orin Nano pilot reports median inference latencies of 0.040–0.046 ms for CPU URL detectors and 0.068 ms for a separately prepared TensorRT FP16 engine. The public workflow lets researchers exercise export and inference without private data. Published pilot aggregates support inspection and rendering; reproducing the detector evaluation requires the original models and test data. TensorRT conversion and service integrations remain outside this release.

Keywords: Edge computing, Phishing detection, Vietnamese, ONNX Runtime, TensorRT, Benchmarking

1. Software purpose

Phishing classifiers intended for local deployment need runtime measurements in addition to predictive accuracy. PhishVN Edge extracts the export and CPU benchmarking tools from the wider PhishVN project [1] into a standalone repository. It supports experiments on Linux edge boards, including NVIDIA Jetson devices, while also allowing a workstation demonstration.

The release accepts a trusted scikit-learn estimator, exports it to ONNX, and measures inference latency and throughput. A small classifier trained on synthetic numeric data makes the workflow executable without access to the research models or private URL records. This demonstration validates the software path; it does not provide a trained Vietnamese phishing detector.

Local filtering is a potential application. SMS forwarding, a scoring API, and gateway integration would require additional software and end-to-end evaluation. The measurements here concern model inference on numeric inputs, excluding parsing, feature extraction, and network overhead.

1.1. Software description

The release contains three scripts. `export_edge.py` loads a trusted scikit-learn estimator or a dictionary containing an estimator, exports it through `skl2onnx`, and performs a short inference timing check. Export failures return an error. The resulting tree-ensemble ONNX model targets ONNX Runtime [2]; it is not automatically a TensorRT-compatible graph.

`bench_edge_ort.py` warms up an ONNX Runtime CPU session, measures batch-one median, p95, and p99 latency, and runs a sustained throughput window. It reports

process peak resident memory. On Jetson, an available `tegrastats` process supplies board-power and temperature samples at 500 ms intervals. Board energy per inference uses measured load power and throughput; marginal energy requires a separate idle measurement. A file-backed telemetry capture prevents pipe-buffer saturation during longer runs.

`make_edge_assets.py` renders the article figure and table from public aggregate snapshots. It does not load private records or rerun model predictions. The repository documents the source of each snapshot and distinguishes historical measurements from newly generated local benchmark output.

1.2. Example workflow

After installing the Python requirements, a user runs `make demo`. This trains a deterministic synthetic classifier with nine input features, exports it to ONNX, and writes local benchmark JSON. Users can then substitute their own trusted model and its input dimension, preserving the feature order used during training.

The integration check compares the exported model's labels and probabilities with the original scikit-learn model, then validates the benchmark output. The article assets can be re-generated independently with `make assets`; `make paper` builds the PDF with pdfLaTeX.

2. Impact overview

The software makes an existing deployment experiment accessible as a small, independently versioned workflow. Its immediate contribution is an executable export-and-measure path and inspectable historical results. Researchers can use it to compare CPU runtimes or device configurations for a fixed estimator and feature schema, before implementing a full filtering service.

Email address: thaivn_ph@utc.edu.vn (Thai Nguyen Vu)

Code metadata

Nr.	Code metadata description	PhishVN Edge
C1	Current code version	v0.1.0
C2	Permanent link to code/repository used for this code version	https://github.com/vuthainguyen1602/phishvn-edge/tree/v0.1.0
C3	Permanent link to Reproducible Capsule	Reproducible capsule link pending
C4	Legal Code License	MIT (code); bundled Elsevier class: LPPL 1.3 or later
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	Python, ONNX Runtime, TensorRT, LaTeX
C7	Compilation requirements, operating environments & dependencies	Python 3; Linux on NVIDIA Jetson Orin Nano; ONNX Runtime for CPU inference; TensorRT for GPU inference. Workstation result generation supports macOS/Linux.
C8	If available Link to developer documentation/manual	https://github.com/vuthainguyen1602/phishvn-edge/blob/v0.1.0/docs/JETSON_DEPLOY.md
C9	Support email for questions	thai_vn_ph@utc.edu.vn

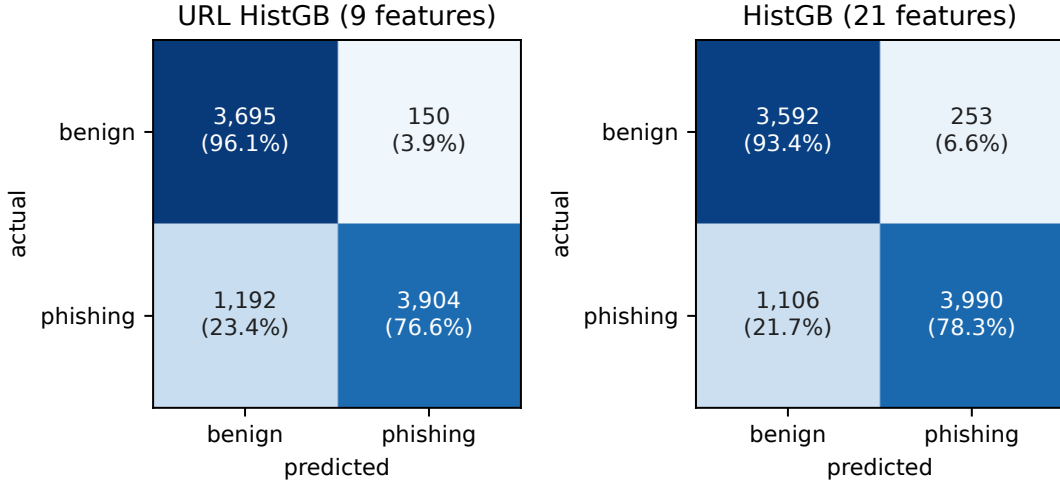


Figure 1: Confusion matrices rendered from archived aggregate counts for the two FP32 URL models on the historical temporal test split ($n = 8,941$ per model). Rendering these counts does not rerun detector evaluation.

The synthetic example removes the need for private research data when checking installation and model conversion. Separating aggregate rendering from measurement also makes clear which outputs can be regenerated from the public repository and which require additional artifacts. No independent user adoption, commercial deployment, or publications enabled by this standalone release have been documented. The linked PhishVN dataset article supplies project context and predates this extraction.

2.1. Pilot measurements

The historical pilot supplies reference measurements on real hardware; it is not rerun by the public demonstration. Table 1 reports the populated Jetson Orin Nano rows from the PhishVN Edge benchmark workflow. For model inference alone, the two URL tree-ensemble detectors reach 21,180–24,121 inferences/s with median latency below 0.05 ms. The TensorRT FP16 GPU path reaches 12,623 inferences/s with lower marginal energy

per inference in this pilot, while its host latency, including transfers, is higher than the CPU rows. This comparison does not isolate the contribution of transfer overhead. The engine was benchmarked separately using TensorRT tools [3].

These measurements are useful beyond the specific detector. They show how small URL models can be dominated by runtime overhead, host-device movement, and platform idle draw, which are often hidden in workstation-only evaluations. They also provide a baseline that future Vietnamese text models, distilled transformers, and INT8 variants must beat before they justify additional complexity on a low-power appliance.

3. Limitations and future work

The initial release includes the export and CPU benchmark path, a synthetic model generator, and historical aggregate snapshots. The original trained URL detectors, private test splits, and raw device logs are not bundled. Consequently, the

Table 1: Historical pilot on NVIDIA Jetson Orin Nano. Board telemetry was sampled at 500 ms. These aggregate rows are not remeasured by the asset renderer.

Model	Runtime	Precision	Median (ms)	Throughput (inf./s)	Marginal energy (mJ)
URL HistGB	ONNX Runtime CPU	FP32	0.046	21,180	0.13
HistGB	ONNX Runtime CPU	FP32	0.040	24,121	0.11
HistGB	TensorRT GPU	FP16	0.068	12,623	0.09

public repository reproduces the software demonstration and aggregate visualisation, but not the original detector evaluation or hardware campaign.

TensorRT engine conversion and FP16 classification validation remain outside this release. Longer thermal soaks, external power-meter checks, neural text models, and INT8 calibration are also pending. Synthetic numeric inputs test inference execution rather than the distribution of real messages or URLs. A complete deployment study would additionally measure feature extraction, request handling, and service behaviour under realistic traffic.

Code availability

Version v0.1.0 is available at <https://github.com/vuthainguyen1602/phishvn-edge/tree/v0.1.0>. The software code is MIT-licensed. Bundled Elsevier class files retain their LPPL license. The repository includes installation instructions, an executable synthetic example, integration checks, and aggregate-result provenance. A public reproducible capsule and an archival software DOI remain pending.

Declarations

Competing interests. The author declares no competing interests.

Funding. No specific funding is declared for this software article.

Ethics. The software is intended for defensive research and local phishing filtering. Dataset collection and release should follow the ethics, privacy, and data-governance statements of the PhishVN project. Private mappings, raw personally identifying information, and live malicious artifacts should not be included in the public software archive.

References

[1] T. Nguyen Vu, “PhishVN: A time-stamped Vietnamese URL phishing dataset with impersonation-scenario labels and confidence tiers,” Mendeley Data, V4, 2026. <https://doi.org/10.17632/b97hxbxtpd.4>. Data article: *Data in Brief*, vol. 68, art. 113195, 2026. <https://doi.org/10.1016/j.dib.2026.113195>.

[2] ONNX Runtime developers, “ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator,” 2018–present. [Online]. Available: <https://github.com/microsoft/onnxruntime>

[3] NVIDIA, “NVIDIA TensorRT Developer Guide,” NVIDIA Corporation. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/>